

Elementary Algorithms

9th Grade Computer Science Recap (C/C++)



Maximum & Minimum Problems

Identifying extreme values in a dataset.

The Maximum Principle

To find the maximum among several numbers (e.g., **a**, **b**, **c**), we don't need to compare every number with all the others simultaneously.

Algorithmic logic (The Assumption Method):

1. Assume the **first number** is the maximum.
2. Compare the second number with the maximum. If it's larger, it becomes the new maximum.
3. Repeat the process for all remaining numbers.

```
// Example: Maximum of a, b, and c
int a = 15, b = 42, c = 8;

// Step 1: Assume 'a' is the largest
int max_val = a;

// Step 2: Update if we find something larger
if (b > max_val) {
    max_val = b;
}
if (c > max_val) {
    max_val = c;
}

cout << "The maximum is: " << max_val;
```

Maximum in a Sequence

When processing a sequence of **N** numbers, we proceed similarly, using a loop structure (**for** or **while**).

- Read the number **N** (how many numbers to process).
- Read the **first number** separately and set it as `max_val`.
- Read the remaining **N-1** numbers in a loop, updating `max_val` at each step if necessary.

```
int n, x, max_val;
cin >> n;

// Read the first element
cin >> x;
max_val = x;

// Process the remaining n-1 elements
for (int i = 1; i < n; i++) {
    cin >> x;
    if (x > max_val) {
        max_val = x;
    }
}
cout << "Max value read: " << max_val;
```

Digits of a Number

Extracting and processing the internal structure of integers.

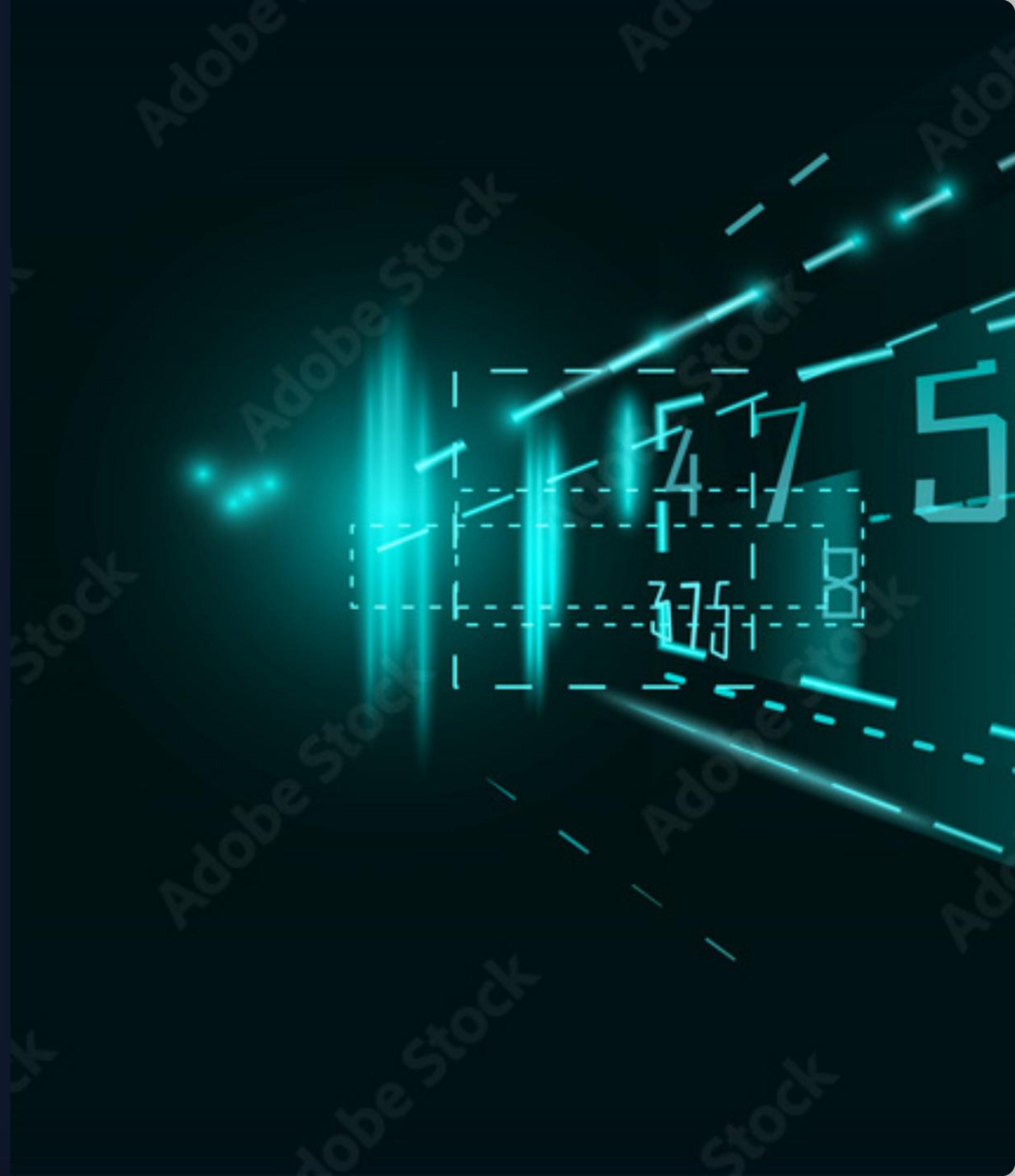
The Logic of Extraction

Everything relies on Base 10.

To disassemble a number digit by digit (from right to left), we use two vital mathematical operations in C++:

Modulo ($\% 10$): Always gives us the last digit of the number.

Division ($/ 10$): Removes the last digit. In C++, integer division automatically truncates the decimal part.



Classic Problems



Sum of Digits

At each step, we extract the last digit using $\% 10$, add it to a variable initialized to 0, and then chop the digit off using $/ 10$.



Count of Digits

Instead of adding the digit's value, we simply increment a counter (e.g., $k++$) every time we chop a digit, until the number becomes 0.



Reversed Number

The reverse is built iteratively: $rev = rev * 10 + digit$. This shifts the old digits to the left and appends the newly extracted one.

Algorithm in C++

Any processing of a number's digits n uses a standard template with a **while** loop.

The stopping condition is $n > 0$ (or $n \neq 0$).

Warning: If you need the initial value of n later in your program, make sure to create a copy (e.g., `int copy_n = n;`) before entering the loop, because n will be reduced to 0.

```
int n, sum = 0, rev = 0;
cin >> n;

while (n > 0) {
    int digit = n % 10; // Extract last digit

    sum += digit; // Add to sum
    rev = rev * 10 + digit; // Build reverse

    n /= 10; // Remove processed digit
}

cout << "Sum: " << sum << endl;
cout << "Reversed: " << rev << endl;
```

Divisors of a Number

Finding divisors, checking for primes, and mathematical optimizations.

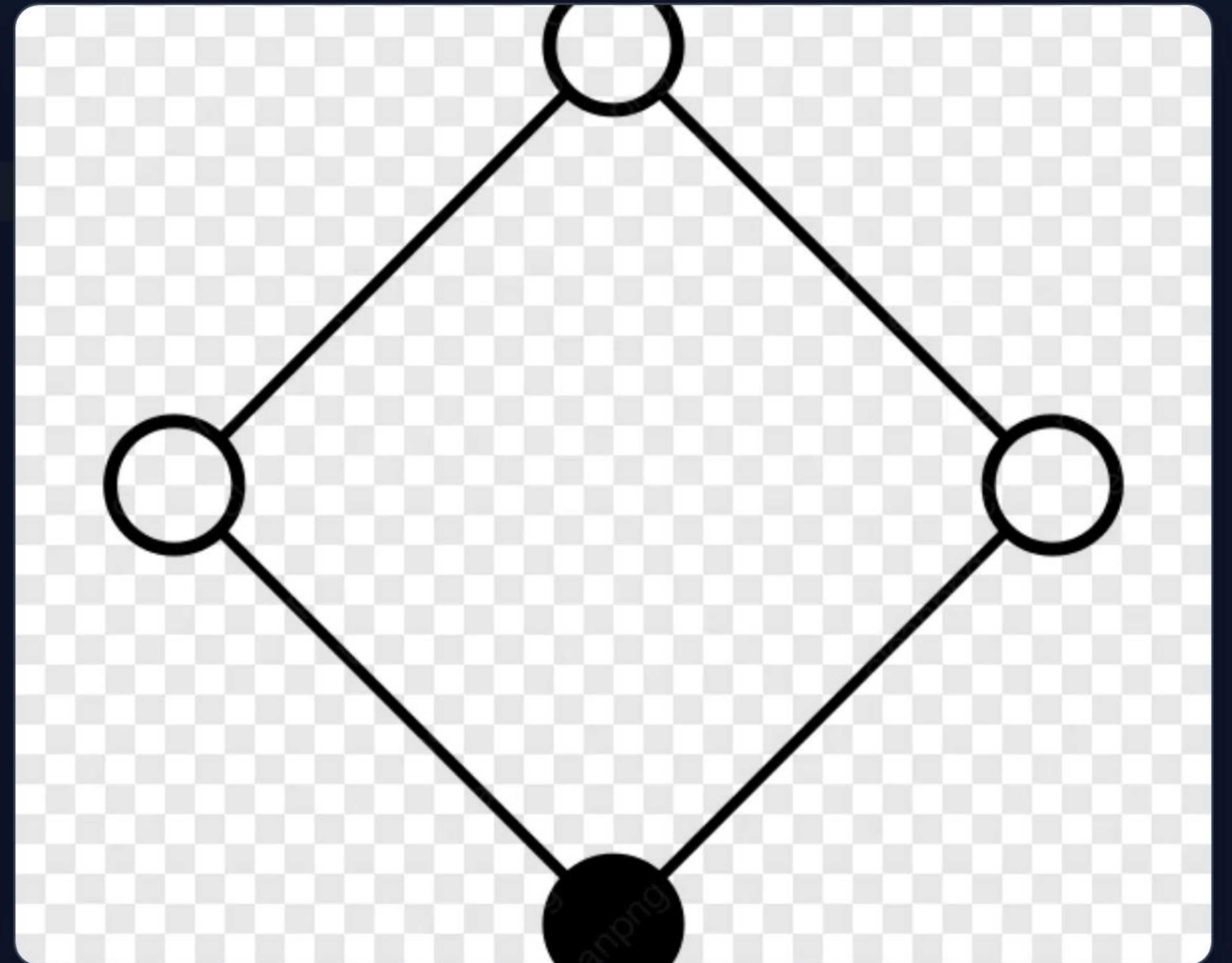
Optimizing Divisors

The naive algorithm checks all numbers from 1 to n . This is extremely slow for very large numbers (e.g., 1 billion).

Key Observation: Divisors always come in pairs (e.g., for 36, we have the pair 4 and 9). The smaller number in a pair can never exceed the square root of n .

Therefore, it's sufficient to search for divisors up to:

$$d \leq \sqrt{n}$$



Primality Test

A number is **prime** if it divides evenly only by 1 and itself. (0 and 1 are NOT prime).

To verify, we assume the number is prime (`bool isPrime = true;`). We then search for any divisor between 2 and the square root of `n`.

If we find at least one divisor, the number is NOT prime, so we set `isPrime = false` and stop the search early using **break**.

```
int n; cin >> n;
bool isPrime = true;

if (n < 2) isPrime = false;

for (int d = 2; d * d ≤ n; d++) {
    if (n % d == 0) {
        isPrime = false; // Found a divisor
        break; // Stop the loop immediately
    }
}

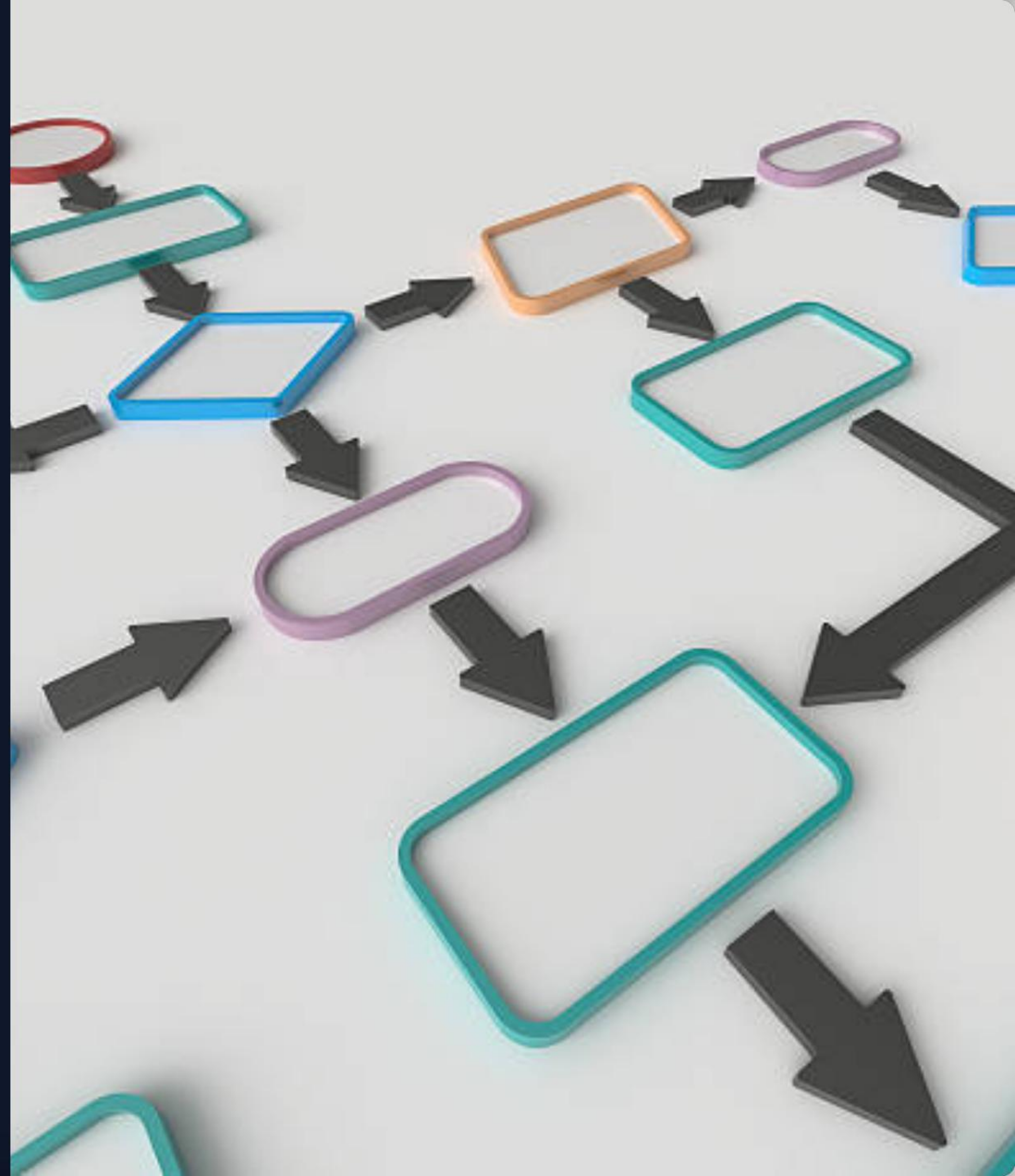
if (isPrime) cout << "Prime";
else cout << "Not prime";
```

Prime Factorization

Fundamental Theorem of Arithmetic.

Every composite number can be written as a unique product of prime numbers raised to specific powers.

The algorithm repeatedly divides the number by the smallest possible divisor (starting with 2) until it can no longer be divided. Then we move to the next divisor, repeating the process until n becomes 1.



Code: Factorization

There's no need to check if d is prime! The inner loop (`while (n % d == 0)`) consumes all factors of d . By the time we reach a multiple of d (like 4 or 6), the number n has already been divided by the prime factors (2, 3) and won't be divisible anymore.

The line `if (d * d > n)` is a **crucial optimization** to avoid iterating through unnecessary numbers up to a very large final prime factor.

```
int n; cin >> n;
int d = 2;
while (n > 1) {
    int p = 0; // Power of factor d
    while (n % d == 0) {
        p++;
        n /= d;
    }

    if (p > 0) {
        cout << d << "^" << p << endl;
    }

    d++;
    if (d * d > n && n > 1) {
        d = n; // Optimize for last large factor
    }
}
```

Questions?

Ready to solve problems in C++!

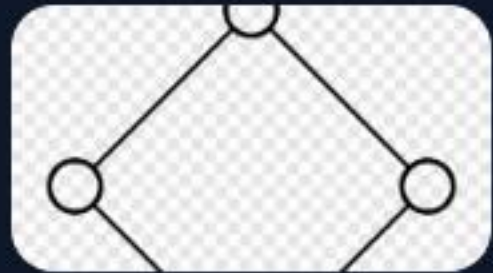
Good luck with the implementation!

Image Sources



https://as1.ftcdn.net/v2/jpg/04/09/90/08/1000_F_409900893_LN2AZzKtCMUILXoWcivrSy79iHROyH4A.jpg

Source: www.europosters.pt



<https://banner2.cleanpng.com/20180625/iaa/kisspng-mathematical-problem-node-mathematics-tree-edge-co-30-minutes-5b31494425c1a8.3512494115299566761547.jpg>

Source: www.cleanpng.com



<https://media.istockphoto.com/id/490245946/photo/flow-chart-diagram-concept.jpg?s=612x612&w=0&k=20&c=SWE8UA7W8oBrMYO2VbXcxJjSdew-Gjl9v5zKknJtl4U=>

Source: www.istockphoto.com